

India Flood, Drought & Water-Scarcity Early Warning System

Hackathon Implementation Plan & Technical Documentation

Team size: 4 | **Time budget:** 24 hours | **Scope:** India, 2-3 demo regions | **Tools:** Claude Opus 4.6, Google Earth Engine

Scope Lock — read this before writing any code

We are building a **risk-index model per grid cell** (tabular ML: XGBoost/Random Forest → a 0-1 risk score, rendered as a colored map cell), **not** a pixel-level raster segmentation model. This is scientifically legitimate (it's how real indices like SPI and flood-susceptibility maps work) and is achievable in 24 hours. Visually, on the map, it will look identical to a "predicted calamity map." If anyone on the team starts trying to train a true image-segmentation model from scratch tonight, stop — that's the single biggest risk to finishing.

Explicitly out of scope for this build: Moon gravity / tidal-level data (relevant only to coastal storm-surge flooding, not the inland river-flood and drought case we're building — revisit only if we add a coastal sub-case later). True pixel-level raster segmentation. Real SMS delivery to real residents (we simulate the trigger + message, with an optional live demo to 1-2 verified test numbers). Live all-India coverage (we go deep on 2-3 regions, not wide on 28 states).

Table of Contents

1. [Project Overview](#)
 2. [System Architecture](#)
 3. [Datasets & APIs — Master Reference](#)
 4. [Google Earth Engine — Setup Guide](#)
 5. [Feature-to-Model Mapping](#)
 6. [Forecast Integration \(7-15 Day Horizon\)](#)
 7. [Alert System Design](#)
 8. [Frontend / UI Plan](#)
 9. [Role Division — 4 People](#)
 10. [Hour-by-Hour Timeline](#)
 11. [Risks & Fallbacks](#)
 12. [Quick Reference — All Links](#)
-

1. Project Overview

Problem: Predict flood and drought/water-scarcity risk across India, visualize it on a Google-Earth-style map with a historical/future timeline slider, and generate resident-facing alerts 7-15 days ahead of a predicted event so people can evacuate or store water.

Core decisions locked in from scoping discussion:

- Two separate models (flood, drought) — different signatures, different features, easier to train and explain than one combined model.
 - Output is a **risk score per grid cell per day** (5–10 km cells), not a segmented image.
 - Pick **2–3 demo regions** with full pipeline + validation, rather than shallow all-India coverage. Suggested: one flood-prone (Assam, Bihar, or Kerala) + one drought-prone (Marathwada, Bundelkhand, or Rayalaseema). Final pick depends on which region's historical label data is cleanest — see Section 3.2.
 - "Real-time" = **recomputed on a schedule** (e.g., daily), not true streaming. This is the honest and expected standard for a hackathon build.
 - Flood forecast horizon (1/3/7/15 days) is driven by an actual weather **forecast** feed (Section 6) — not just historical data, which cannot predict the future by itself.
 - Drought "future" is framed honestly as **trend + climatology projection**, since skillful 60–90 day drought forecasts don't really exist even in production systems. Say this plainly in the pitch — judges respect honesty about model limits more than a hand-wave.
-

2. System Architecture



Precompute risk for each demo region for a range of dates (past + projected future) and store as static JSON/GeoJSON per day. The timeline slider then just swaps which precomputed layer is displayed — this is far more demo-reliable than live inference on every slider drag, and just as impressive to watch.

3. Datasets & APIs — Master Reference

3.1 Rainfall — read this one carefully, it has a gotcha

Purpose	Source	ID / Endpoint	Resolution & Cadence	Notes
Historical daily rainfall, 3-day/7-day accumulation, long-term climatology for anomaly baselines	CHIRPS (GEE)	<code>UCSB-CHG/CHIRPS/DAILY</code> (or newer <code>UCSB-CHC/CHIRPS/V3/DAILY_SAT</code>)	0.05° (~5.5 km), daily, 1981-present	Best all-round source for historical accumulation features and drought rainfall-deficit calcs.
Live / today's rainfall + 15-day forecast	Open-Meteo Forecast API	<code>api.open-meteo.com/v1/forecast</code>	~1-11 km depending on model, hourly	This is your actual "live" and "future" rainfall source — free, no key, no rate-limit headaches. See Section 6.
Sub-daily rainfall (1h/3h/6h) for historical training data	GPM IMERG (GEE)	<code>NASA/GPM_L3/IMERG_V07</code>	0.1° (~11 km), half-hourly	⚠ The Final-Run product this collection is built on has ~3.5 month latency — it is NOT suitable for "live" pulls. Use it only to engineer sub-daily features in your <i>historical training set</i> ; use Open-Meteo hourly data for anything that needs to be current.

3.2 Soil Moisture

Purpose	Source	ID	Resolution & Cadence
Surface and root-zone soil moisture in one dataset (covers flood Tier-1 #5 <i>and</i> drought Tier-2 root-zone moisture)	NASA SMAP L4 (GEE)	<code>NASA/SMAP/SPL4SMGP/008</code> , bands <code>sm_surface</code> , <code>sm_rootzone</code>	~9–11 km, every 3 hours

3.3 Terrain & Hydrology (mostly static — compute once, reuse)

Purpose	Source	ID	Notes
Elevation	SRTM (GEE)	<code>USGS/SRTMGL1_003</code>	30 m, static
Slope	Computed from SRTM	<code>ee.Terrain.slope(dem)</code>	No separate dataset needed — one line of code on the elevation image.
Flow accumulation (<i>also covers "upstream catchment area" — the flow-accumulation value at a point already is the upstream drainage area</i>)	HydroSHEDS (GEE)	<code>WWF/HydroSHEDS/15ACC</code> (15 arc-sec, ~450 m) or <code>30ACC</code> (~900 m, lighter)	Static.
Drainage direction (used to derive drainage density / river extraction)	HydroSHEDS (GEE)	<code>WWF/HydroSHEDS/15DIR</code>	Static.

River network (vector, for distance-to- river)	HydroSHEDS (GEE)	WWF/HydroSHEDS/v1/FreeFlowingRivers	Compute distance-to- river as distance from each cell centroid to the nearest line feature, or threshold flow accumulation to build a river mask and run a distance transform.
Watershed boundaries	HydroSHEDS (GEE)	WWF/HydroSHEDS/v1/Basins/hybas_5 (levels 1-12 available, 5 is a reasonable mid-scale default)	Static.

3.4 Vegetation, Temperature & Evapotranspiration

Purpose	Source	ID	Resolution & Cadence
NDVI/ EVI (current value + anomaly vs. climatology)	MODIS (GEE)	MODIS/061/MOD13Q1	250 m, 16- day composite
Land surface temperature (temperature anomaly, LST)	MODIS (GEE)	MODIS/061/MOD11A1	1 km, daily
Evapotranspiration	MODIS (GEE)	MODIS/061/MOD16A2GF (gap-filled — use this, not the non-GF MOD16A2 which only covers 2021+)	500 m, 8- day, 2000- present

3.5 SAR, Water Extent & Land Cover

Purpose	Source	ID	Resolution & Cadence	Notes
Flood extent detection	Sentinel-1 SAR GRD	COPERNICUS/S1_GRD, IW mode, VV/VH	10 m, ~6- day revisit	SAR penetrates

(before/after change)	(GEE)			cloud cover — critical because India's floods happen during monsoon, exactly when optical satellites are clouded out. Standard method: compare a pre-flood and post-flood mosaic (Otsu-threshold the difference) — this is a well-documented technique (search "UN-SPIDER Sentinel-1 flood mapping recommended practice" for the reference workflow).
Existing / historical water extent baseline	JRC Global Surface Water (GEE)	JRC/GSW1_4/GlobalSurfaceWater (occurrence band) + JRC/GSW1_4/MonthlyHistory	30 m, monthly, 1984-2021	Good for "what's normally water here" baseline to compare SAR detections against.
NDWI	Computed from Sentinel-2 SR	COPERNICUS/S2_SR_HARMONIZED, bands B3 (green) & B8 (NIR) → $(\text{Green} - \text{NIR}) / (\text{Green} + \text{NIR})$	10 m	Not a standalone dataset — one line of band math.
Land cover,	Dynamic	GOOGLE/DYNAMICWORLD/V1	10 m,	9 classes

urban/built-up %	World (GEE)		updates every 2–5 days	including built and water — near-real-time, better than static land cover for this use case.
Groundwater / storage trend	GRACE (GEE)	NASA/GRACE/MASS_GRIDS_V04/LAND	⚠ Very coarse (~300 km cells), monthly	Useful only as a regional/basin-level trend indicator, not a per-cell feature. Say this plainly if you use it.

3.6 India-specific historical labels — the hardest problem, mostly solved

Historical flood/drought **labels** were flagged as the hardest data problem going in. It turns out there are excellent, ready-to-use, India-specific sources — lead with these before falling back to hand-curation.

Need	Source	Link	What it gives you
Flood labels — best option	India Flood Inventory-Impacts (IFI-Impacts), IIT Delhi	zenodo.org/doi/10.5281/zenodo.4742142	IMD-sourced flood events 1967–2023, includes a ready District Flood Severity Index CSV — this alone can be your flood label column.
Flood labels — event database with catchment attributes	INDOFLOODS, IIT Delhi	zenodo.org/records/14584655	Discharge, peak level, event duration + catchment attributes per event, DOI-cited.

Flood labels — long-term simulated flood extent	India Flood Atlas, IIT Gandhinagar	indiafloodatlas.in · data: github.com/wcl-iitgn/india-flood-atlas-data	10 km gridded annual max flooded area, 1901–2020.
Flood labels — official state atlases (PDF, needs manual digitizing)	NDMA Flood Hazard Atlases	ndma.gov.in/flood-hazard-atlases	State-level flood hazard maps, validated by state disaster authorities — good for cross-checking, not bulk ML-ready.
Flood labels — global event archive with GIS/GeoTIFF maps	Dartmouth Flood Observatory	floodobservatory.colorado.edu	Individual India flood events since 1985 with downloadable extent maps — good for validating specific historical events in your demo narrative.
Drought labels — best option, and it's live	India Drought Monitor, IIT Gandhinagar	indiadroughtmonitor.in	Live, weekly, district-level Combined Drought Index (5-class: D0 abnormally dry → D4 exceptional), downloadable, archive from July 2021–present. Doubles as ground truth <i>and</i> as a way to sanity-check

			your own model's output side-by-side.
Drought labels — long-term climatology for anomaly baselines	India Drought Atlas, IIT Gandhinagar	github.com/wcl-iitgn/india-drought-atlas-data	0.05° gridded monthly precipitation & temperature, 1901–2021 — exactly what you need to compute "how abnormal is this" anomalies.
Bonus — reservoir/dam context, if you want it	GeoID (Geoportal of Indian Dams), IIT Gandhinagar	github.com/wcl-iitgn/geoid	5,400+ Indian dams with catchment characteristics, land cover, and flood risk assessment.
Live validation (not bulk historical, but good for your pitch's "would this have caught it" moment)	CWC Flood Forecast Dashboard	aff.india-water.gov.in · cwc.gov.in/ffm_dashboard	Official 5–7 day flood advisories for 20 major river basins.

Recommendation: Pick your 2–3 demo regions partly based on where these label sources have the best coverage — the Ganga/Brahmaputra basin states (Assam, Bihar) are unusually well-served by both IFI-Impacts and the India Flood Atlas.

4. Google Earth Engine — Setup Guide

⚠ Do this first, at hour 0, for whoever will run the data-pulling scripts. GEE now requires a Google Cloud Project registration step (not just a simple sign-up) — it's fast, but don't discover the extra step at hour 10.

1. **Sign up / register a Cloud Project.** Go to earthengine.google.com/signup, sign in with a Google account, and follow the prompt to create (or select) a Google Cloud Project. Full walkthrough if anything is unclear: developers.google.com/earth-engine/guides/access.

2. **Register the project as noncommercial** (research/education/hackathon use qualifies — it's free). Access is enabled immediately after registration, per Google's own documentation. Using an institutional/college email can smooth eligibility checks if prompted. Background: earthengine.google.com/noncommercial.
3. **Two ways to work — pick per person:**
 - **Code Editor (zero install, browser only):** code.earthengine.google.com — JavaScript, runs in-browser, nothing to authenticate locally. Good for whoever wants to prototype fast without fighting local Python auth.
 - **Python (for the actual pipeline / CSV export):**

```
bash
```

```
pip install earthengine-api --upgrade
```

```
python
```

```
import ee
ee.Authenticate()          # opens a browser once, stores a token local
ee.Initialize(project='YOUR-CLOUD-PROJECT-ID')
```

```
Docs: [python_install](https://developers.google.com/earth-
engine/guides/python_install) · [auth guide]
(https://developers.google.com/earth-engine/guides/auth)
```

4. Test it works:

```
python
```

```
img = ee.Image('USGS/SRTMGL1_003')
print(img.getInfo()['bands'][0])    # should print elevation band metadata
```

Starter script — the core pattern everyone will reuse

This is a skeleton, not tested production code — it demonstrates the pattern (define area → build a grid → pull + stack features → reduce to per-cell values → export). Adapt and debug it against your actual region and dataset list.

```
python
```

```

import ee
ee.Initialize(project='YOUR-CLOUD-PROJECT-ID')

# 1. Area of interest – example bounding box, replace with your demo region
aoi = ee.Geometry.Rectangle([89.7, 24.1, 96.0, 28.2]) # example: Assam

# 2. Build a grid of cells over the AOI (adjust EPSG to a UTM zone covering
proj = ee.Projection('EPSG:32646').atScale(5000) # ~5km cells
grid = aoi.coveringGrid(proj)

# 3. Pull a rainfall feature: 7-day accumulated CHIRPS rainfall ending today
end = ee.Date(ee.Date.now())
start = end.advance(-7, 'day')
rain_7d = (ee.ImageCollection('UCSB-CHG/CHIRPS/DAILY')
           .filterDate(start, end).sum().rename('rain_7d_mm'))

# 4. Soil moisture (surface + root zone, most recent available)
soil = (ee.ImageCollection('NASA/SMAP/SPL4SMGP/008')
        .sort('system:time_start', False).first()
        .select(['sm_surface', 'sm_rootzone']))

# 5. Terrain features
dem = ee.Image('USGS/SRTMGL1_003').select('elevation')
slope = ee.Terrain.slope(dem)
flow_acc = ee.Image('WWF/HydroSHEDS/15ACC').select('b1').rename('flow_acc')

# 6. Stack everything into one multi-band image
features = rain_7d.addBands([soil, dem, slope, flow_acc])

# 7. Reduce to one row per grid cell
cell_features = features.reduceRegions(
    collection=grid, reducer=ee.Reducer.mean(), scale=250
)

# 8. Export – or use geemap.ee_to_pandas(cell_features) to pull straight into
task = ee.batch.Export.table.toDrive(
    collection=cell_features, description='flood_features_export', fileFormat='CSV'
)
task.start()

```

Repeat step 3–5's pattern for every dataset in Section 3, add each as a band in step 6, and you have your full feature table.

5. Feature-to-Model Mapping

5.1 Flood model

Feature	Source (see Section 3)	Note
Rainfall 1h/3h/6h/24h	GPM IMERG (historical) + Open-Meteo (live)	
Rainfall 3-day / 7-day accumulation	CHIRPS	
Rainfall anomaly	CHIRPS vs. India Drought Atlas climatology	
Soil moisture	SMAP L4	
Elevation, slope	SRTM	Static — compute once
Distance to river, flow accumulation	HydroSHEDS	Static — compute once
Historical flood label	IFI-Impacts DFSI / India Flood Atlas	See 3.6
Sentinel-1 SAR, existing water extent	Sentinel-1 + JRC GSW	
NDWI	Sentinel-2 band math	
Land cover, urban %	Dynamic World	
Upstream catchment area	HydroSHEDS flow accumulation	Same dataset as flow accumulation
Temperature, evapotranspiration	MODIS LST, MOD16A2GF	
Historical flood frequency	Derived: count of past flood labels per cell	

5.2 Drought model

Feature	Source (see Section 3)	Note
Rainfall anomaly, 7/30/60/90-day deficit	CHIRPS vs. India Drought Atlas climatology	
Soil moisture (surface + root-zone)	SMAP L4	Same dataset serves both
NDVI anomaly	MODIS MOD13Q1 vs. climatology	
Temperature anomaly, LST	MODIS MOD11A1	
Dry-spell duration	Derived from CHIRPS daily (consecutive dry days)	
Historical drought label	India Drought Monitor (2021-present)	Live + historical in one source
Evapotranspiration	MOD16A2GF	
Land cover	Dynamic World	
Groundwater/storage indicator	GRACE	Coarse — regional trend only
Historical drought frequency	India Drought Monitor archive	
Historical rainfall/NDVI climatology	India Drought Atlas	Static — compute once

Modeling approach: XGBoost or Random Forest classifier/regressor per model, trained on the grid-cell feature tables above with the historical labels as targets. This is a same-day-trainable approach on a laptop — no GPU cluster required, which matters given the timeline.

6. Forecast Integration (7-15 Day Horizon)

This is where "current conditions" data (Section 3) stops being enough — none of it predicts the future by itself. You need an actual forecast feed.

API	Endpoint	What it gives you	Notes
Open-Meteo Forecast API	<code>api.open-meteo.com/v1/forecast</code>	Hourly/daily precipitation forecast up to 16 days, free, no key	This is your primary "future rainfall" signal.
Open-Meteo Flood API	<code>open-meteo.com/en/docs/flood-api</code>	Daily river discharge forecast (GloFAS model), up to ~210 days out, ~5 km resolution, free, no key	Arguably even more useful than raw rainfall for flood risk — it's already routed through a hydrological model. Worth using directly as a feature for your 1/3/7/15-day flood projection.

Flood forecast pipeline: pull forecasted rainfall + forecasted river discharge for a cell → feed into the trained flood risk model alongside current soil saturation → get a projected risk score for day 1/3/7/15.

Drought "forecast": be upfront in the pitch that this is a **trend + climatology projection** (current deficit trajectory compared against the India Drought Atlas climatology), not a hard forecast — skillful 60–90 day drought forecasting doesn't really exist in operational systems either. Judges will respect the honesty.

7. Alert System Design

Trigger logic (starting proposal — tune thresholds against your validation data):

Risk Level	Days to Event	Suggested Action
 Low	—	No alert
 Moderate	7-15 days out	Early advisory — monitor conditions
 High	3-7 days out	Prepare — store water, move valuables to higher ground
 Severe	0-3 days out	Evacuate / immediate action

Implementation for the hackathon: generate the alert record (region, risk level, days-to-event, recommended action) and display it in a notification panel in the UI — this is what a judge needs to see to evaluate the logic. A live SMS demo is a nice-to-have on top, not the deliverable itself.

If you want one live SMS demo:

- Twilio ([twilio.com](https://www.twilio.com)) — \$15 free trial credit (~1,500 messages). ⚠️ Trial accounts can only send to phone numbers you've manually verified in the console — plan to demo to 1-2 team members' phones, not a real resident list.
 - For India-only pricing at real scale later, providers like MSG91 are meaningfully cheaper than Twilio's USD-denominated rates — not needed for the hackathon demo, worth knowing for the "what's next" slide.
-

8. Frontend / UI Plan

- **Map library:** [Mapbox GL JS](#) (free tier: 50,000 map loads/month) is the closer match to your "Google Earth" visual goal — supports satellite basemaps and smooth zoom/pan out of the box. Needs a free signup + access token. [Leaflet](#) is the safe, zero-friction fallback if token setup eats time — fully open source, no signup.
 - **Risk overlay:** colored grid/choropleth layer on top of the base map — one color per risk level (see Section 7's palette).
 - **Timeline slider:** swaps which **precomputed** day's risk layer is displayed. Precompute risk for each region for a range of past + projected-future dates and store as static JSON/GeoJSON per day — don't do live inference on every slider drag, it's slower and more failure-prone live.
 - **Click-to-inspect panel:** clicking a cell shows its risk score, days-to-event estimate, and the alert message that would be sent.
-

9. Role Division — 4 People

Person	Primary Ownership (Hr 0-10)	Then (Hr 10-16)	Then (Hr 16-24)
A — Data & Geo-Pipeline Lead	Owns GEE account/project setup for the team. Builds the grid + the core data-pulling script (rainfall, soil moisture, terrain, land cover) producing one clean feature CSV per region. This is the shared artifact B and C consume.	Moves to backend API — serves precomputed risk grids + alert records to the frontend.	Integration testing, demo data prep.
B — Flood Model Lead	Sources flood labels (IFI-Impacts DFSI, India Flood Atlas) in parallel with A's pipeline coming online. Feature-engineers and trains the flood XGBoost/RF model; validates against a known historical event (e.g., a past Assam flood).	Integrates Open-Meteo rainfall + Flood API discharge forecast for the 1/3/7/15-day flood projection.	Polish, owns the flood half of the demo narrative.
C — Drought Model Lead	Sources drought labels (India Drought Monitor, Drought Atlas). Feature-engineers and trains the drought model.	Builds the alert trigger logic (Section 7) — natural fit since this is the risk-score consumer.	Polish, owns the drought half of the demo narrative.
D — Frontend/UI Lead	Starts the map + timeline slider scaffold immediately using mock data — doesn't need to wait for the real pipeline.	Builds out the colored risk overlay, click-to-inspect panel; integrates real data from A's API as it comes online.	Visual polish, leads demo rehearsal (knows the full user-facing flow best).

10. Hour-by-Hour Timeline

Hours	Whole Team	A	B	C	D
0-2	Lock demo regions, GEE account setup (all)	Grid + pipeline skeleton	Source flood labels	Source drought labels	Map scaffold w/ mock data
2-8	—	Feature CSV pipeline running	Flood feature engineering + training	Drought feature engineering + training	Timeline slider + overlay rendering
8-10	Sync: models + pipeline stable?	Starts backend API	Forecast integration (flood)	Starts alert trigger logic	Wires real API into map
10-16	Full integration push	Backend API complete	Validate flood model vs. real event	Alert logic complete	Click panel, visual polish
16-22	End-to-end test on demo regions only	Support/fix	Flood demo narrative	Drought demo narrative	Full demo rehearsal
22-24	Buffer — expect something to break	—	—	—	—

11. Risks & Fallbacks

Risk	Mitigation
GEE registration/verification friction at hour 0	Do it first thing, use an institutional email if available; Code Editor needs only a browser, no local auth.
GPM IMERG "live" data is actually 3.5 months stale (Final Run latency)	Use Open-Meteo for anything that needs to be current; GPM only for historical training features.
Historical label data is messier than expected for your chosen region	Fall back to a smaller hand-curated set for 1 region rather than blocking the whole pipeline — see Section 3.6 for the best starting sources.
GEE compute/export limits on large regions	Keep demo regions small; use <code>scale</code> and <code>maxPixels</code> parameters; export at coarser resolution if exports time out.
Frontend blocked waiting on real data	Build against mock data from hour 0 (already reflected in the timeline).
Twilio trial restrictions block your SMS demo	Verify 1-2 test numbers in the Twilio console well before the demo slot, not during it.

12. Quick Reference — All Links

Google Earth Engine

- Sign up: <https://earthengine.google.com/signup/>
- Access/registration guide: <https://developers.google.com/earth-engine/guides/access>
- Noncommercial use info: <https://earthengine.google.com/noncommercial/>
- Code Editor: <https://code.earthengine.google.com>
- Python install: https://developers.google.com/earth-engine/guides/python_install
- Auth guide: <https://developers.google.com/earth-engine/guides/auth>

Forecast APIs

- Open-Meteo Forecast: <https://open-meteo.com/en/docs>
- Open-Meteo Flood API (GloFAS): <https://open-meteo.com/en/docs/flood-api>

India-specific historical data

- India Flood Atlas: <https://indiafloodatlas.in/>
- India Flood Atlas data repo: <https://github.com/wcl-iitgn/india-flood-atlas-data>
- IFI-Impacts (flood labels, IIT Delhi): <https://zenodo.org/doi/10.5281/zenodo.4742142>
- INDOFLOODS (IIT Delhi): <https://zenodo.org/records/14584655>

- India Drought Monitor: <https://indiadroughtmonitor.in/>
- India Drought Atlas data repo: <https://github.com/wcl-iitgn/india-drought-atlas-data>
- GeoID (Indian dams): <https://github.com/wcl-iitgn/geoid>
- NDMA Flood Hazard Atlases: <https://ndma.gov.in/flood-hazard-atlases>
- Dartmouth Flood Observatory: <https://floodobservatory.colorado.edu/>
- CWC Flood Forecast Dashboard: <https://aff.india-water.gov.in/> and https://cwc.gov.in/ffm_dashboard

Alerts & Frontend

- Twilio: <https://www.twilio.com/>
 - Mapbox: <https://www.mapbox.com/>
 - Leaflet: <https://leafletjs.com/>
-

Open questions for the team to nail down at hour 0: final pick of the 2–3 demo regions, and confirmation everyone can get GEE access without institutional-email friction.